EECE 7205: Computing Fundamentals for Engineers

Spring 2026

Assignment #01: Sorting Algorithms

Student Name	Catalina Buitano
NUID	
Email	buitano.c@northeastern.edu
Instructor	Dr. Nafa
Due Date	Fall 2026

Problem 1: Bubble Sort

Goal: The goal here is to sort A = {34, 7, 23, 32, 5, 62, 14, 19} into increasing order using Bubble Sort, and to show the array after every pass.

Inputs: The input is the array A and its size $n = 8$. The program also runs on an already sorted copy of A.

Outputs: The array after every pass, the sorted array, and the total number of comparisons, swaps, and passes.

Constraints: No `std::sort` or other library sorting function. The program must have `printArray` and `bubbleSort`, and all array printing must go through `printArray`. Counters are passed by reference.

Assumptions: One assumption is that the array is sorted in place, and that equal values are not swapped.

Edge Cases:

1. Array is already sorted (best case)
2. Array is in reverse order (worst case)
3. Duplicate values
4. Array with 0 or 1 elements

Designing Solution: Bubble Sort compares each pair of neighbors and swaps them if the left one is bigger. Each pass moves the largest remaining number to the end, so the inner loop gets one shorter every pass. The swapped flag tracks if a pass made any swaps. If a pass makes none, the array is sorted and the loop stops early. `printArray` is called after every pass to show the array.

Pseudocode:

Input: An array of numbers *arr* and its size n.
Output: Array *arr* sorted in increasing order.

1: $comparisons \leftarrow 0$, $swaps \leftarrow 0$, $passes \leftarrow 0$
2: **for** $pass \leftarrow 0$ **to** $n - 2$ **do**
3: $\quad swapped \leftarrow$ False
4: $\quad$ **for** $j \leftarrow 0$ **to** $n - pass - 2$ **do**

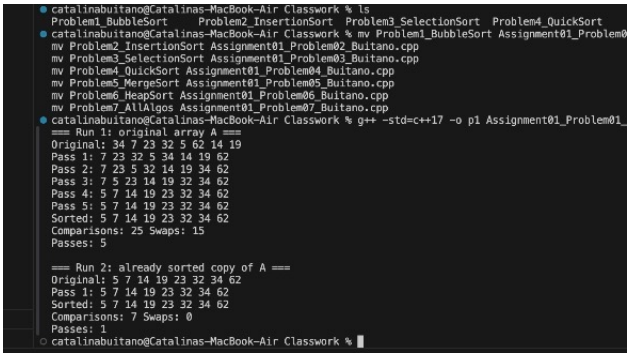


Figure 1: Original Sorted Array and Sorted Copy of Array

```
        comparisons ← comparisons + 1
        if arr[j] > arr[j + 1] then
            Swap(arr[j], arr[j + 1])
            swaps ← swaps + 1
            swapped ← True
        end if
    end for
    passes ← passes + 1
    Print "Pass" passes, printArray(arr, n)
    if swapped = False then
        break
    end if
end for
```

Screenshots:

Explanation: Each pass compares every pair of neighbors in the unsorted part and swaps them if the left one is bigger, so the largest remaining number moves to the end. The sorted part grows on the right by one number every pass, and the inner loop gets one shorter each time. If a pass makes no swaps, every pair of neighbors is already in order, so the loop stops early. Equal values are never swapped because the test uses $>$, so duplicates keep their order. An array with 0 or 1 elements never enters the loop.

Correctness: After each pass, the largest remaining number moves to the end and is in its final spot, so after at most $n - 1$ passes the whole array is sorted. If a pass makes no swaps, every pair of neighbors is in order, so stopping early is okay. Equal numbers are never swapped because the check is `arr[j] > arr[j + 1]`, so Bubble Sort is stable and an array with 0 or 1 elements is left unmodified.

Time & Space Complexity: The best case is $O(n)$, when the array is already sorted: the first pass makes $n - 1$ comparisons and no swaps, so the swapped flag stops the loop after one pass. The average and worst cases are $O(n^2)$, with up to $n(n - 1)/2$ comparisons and swaps for a reversed array. The space complexity is $O(1)$, since the sort is done in place.

Run your program again on an already sorted copy of A. How many passes and comparisons does it make now? Explain why in your report: On the already sorted copy, my program makes 1 pass and 7 comparisons but 0 swaps. On the original Array, it made 5 passes, 25 comparisons, and 15 swaps. In the sorted copy, every pair of neighbors is already in order, so

during pass 1, all 7 comparisons find nothing to swap, and the swapped flag stays false. At the end of pass 1, the program sees that no swaps happened, knows the array is sorted, and stops. It still needs one full pass of $n - 1 = 7$ comparisons to check the order. Without the flag, it would keep going for all 7 passes and 28 comparisons even though nothing changes.

Problem 2: Insertion Sort

Goal: The goal here is to sort A into increasing order using Insertion Sort, showing the key and the array at every step.

Inputs: The input is the array A and its size $n = 8$. The program also runs on the same numbers in increasing order and in decreasing order.

Outputs: The key and the array after each insertion, the sorted array, and the total number of comparisons and shifts.

Constraints: No `std::sort` or other library sorting function. The program must have `printArray` and `insertionSort`, and all array printing must go through `printArray`. Counters are passed by reference.

Assumptions: A comparison is one check of `arr[j] > key`, and a shift is one element moved one position to the right.

Edge Cases:

1. Array is already sorted (fewest shifts)
2. Array is in reverse order (most shifts)
3. Duplicate values
4. Array with 0 or 1 elements

Designing Solution: Insertion Sort builds a sorted part on the left one number at a time. It takes the next number (the key) and shifts every bigger number in the sorted part one spot right, then puts the key in the gap. The inner loop stops as soon as it finds a number that is not bigger than the key. `printArray` is called after each step.

Pseudocode:

Input: An array of numbers *arr* and its size n.
Output: Array *arr* sorted in increasing order.

```
1:  comparisons ← 0, shifts ← 0
2:  for i ← 1 to n − 1 do
3:      key ← arr[i]
4:      j ← i − 1
5:      while j ≥ 0 do
6:          comparisons ← comparisons + 1
7:          if arr[j] > key then
8:              arr[j + 1] ← arr[j]
9:              shifts ← shifts + 1
10:             j ← j − 1
11:         else
12:             break
```

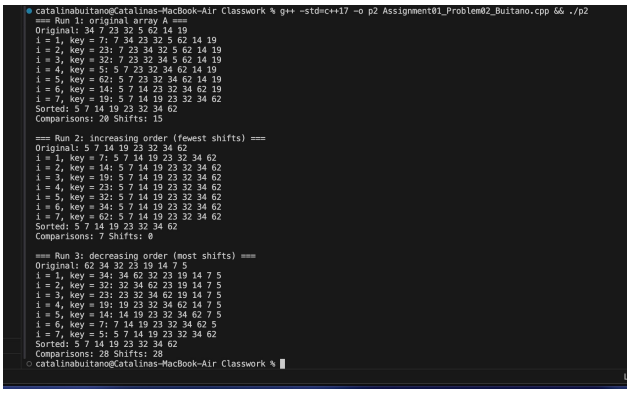


Figure 2: Original Array, Increasing Order, Decreasing Order

13: **end if**
14: **end while**
15: $arr[j+1] \leftarrow key$
16: Print "i =" i, "key =" key, PRINTARRAY(arr, n)
17: **end for**

Screenshots:

Explanation: For each i from 1 to $n-1$, the code saves arr[i] as the key and scans left from $j = i - 1$. Each check of arr[j] > key counts as one comparison. If arr[j] is bigger, it moves one spot to the right, which counts as one shift, and j goes down by one; otherwise the loop stops. The key is then placed at arr[j + 1], and the code prints i, the key, and the array with printArray.

Correctness: Before each step, the left part of the array is already sorted. The key is inserted after every number smaller than or equal to it, so the left part stays sorted, and after the last step the whole array is sorted. Equal numbers are never shifted past each other because the check is `arr[j] > key`, so Insertion Sort is stable, and an array with 0 or 1 elements is left unmodified.

Time & Space Complexity: The best case is $O(n)$, when the array is already sorted and each key needs only one comparison. The average and worst cases are $O(n^2)$, with up to $n(n-1)/2$ comparisons and shifts for a reversed array. The space complexity is $O(1)$, since the sort is done in place.

Which ordering of the same eight numbers makes Insertion Sort do the fewest shifts, and which makes it do the most? Run your program on both orderings, report the counts, and explain the result: Increasing order 5 7 14 19 23 32 34 62 gives 0 shifts and 7 comparisons aka fewest shifts. In increasing order, every key is bigger than the number to its left. Each step makes one comparison, finds nothing to move, and leaves the key in place, so there are 0 shifts and 7 comparisons. On the other hand decreasing order 62 34 32 23 19 14 7 5 gives 28 shifts and 28 comparisons aka most shifts. Every number is smaller than everything before it, so each one has to move all the way to the front. That adds up to 28 shifts and 28 comparisons.

Problem 3: Selection Sort

Goal: The goal here is to sort A into increasing order using Selection Sort, showing the minimum found and the array after each pass.

Inputs: The input is the array A and its size $n = 8$.

Outputs: The minimum value and its index in each pass, the array after each swap, the sorted array, and the total number of comparisons and swaps.

Constraints: No `std::sort` or other library sorting function. The program must have `printArray` and `selectionSort`, and all array printing must go through `printArray`. A swap is only made when minIdx does not equal i.

Assumptions: If there are several equal minimums, the first one found is used.

Edge Cases:

1. Array is already sorted (no swaps needed)
2. Minimum is already in position
3. Duplicate values
4. Array with 0 or 1 elements

Designing Solution: Each pass finds the smallest number in the unsorted part and swaps it to the front of that part. If the smallest is already in place, no swap is made. `printArray` is called after each pass.

Pseudocode:

Input: An array of numbers *arr* and its size n.
Output: Array *arr* sorted in increasing order.

```
1:  comparisons ← 0, swaps ← 0
2:  for i ← 0 to n − 2 do
3:      minIdx ← i
4:      for j ← i + 1 to n − 1 do
5:          comparisons ← comparisons + 1
6:          if arr[j] < arr[minIdx] then
7:              minIdx ← j
8:          end if
9:      end for
10:     Print "Pass" i + 1, "min =" arr[minIdx], "at index" minIdx, "->"
11:     if minIdx ≠ i then
12:         Swap(arr[i], arr[minIdx])
13:         swaps ← swaps + 1
14:     end if
15:     printArray(arr, n)
16: end for
```

Screenshots:

Explanation: For each i from 0 to $n - 2$, the inner loop scans from $i + 1$ to the end, counts each comparison, and keeps the index of the smallest number in minIdx. The code prints the minimum and its index, swaps it into position i only if minIdx $\neq i$, counts the swap, and then prints the array with printArray.

Correctness: Each pass finds the smallest number in the unsorted part and puts it in its final spot, so after $n - 1$ passes the last number is the largest and the array is sorted. Skipping the swap when minIdx = i only avoids swapping a number with itself.

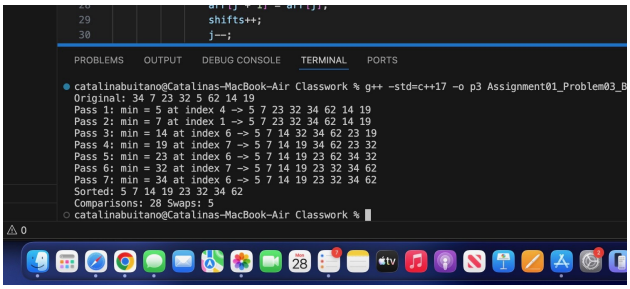


Figure 3: Given Array Input

Time & Space Complexity: The time complexity is $O(n^2)$ in the best, average, and worst case, since every pass scans the whole unsorted part for $n(n-1)/2$ comparisons no matter what the input is. It makes at most $n-1$ swaps. The space complexity is $O(1)$, since the sort is done in place.

Compare the number of swaps with your result for Bubble Sort in Problem 1. Which algorithm makes fewer swaps on A, and why?: Selection Sort makes 5 swaps, and Bubble Sort makes 15 swaps, so Selection Sort makes fewer. Selection Sort finds the smallest number in the unsorted part and swaps it directly into its final spot, so it makes at most one swap per pass. In my output, passes 2 and 7 made no swap because the minimum was already in place, which leaves 5 swaps. Bubble Sort can only swap two neighbors at a time, so each number moves one step per swap. A has 15 pairs of numbers that are out of order, and each Bubble Sort swap fixes only one pair, so it needs 15 swaps.

Problem 4: Quick Sort

Goal: Sort A into increasing order using Quick Sort with the last element as the pivot.

Inputs: Array A with low $= 0$ and high $= n-1$, plus an already sorted copy of A.

Outputs: i, j, and the array after every swap in the first partition; the pivot's final position; the pivot and subarray for every recursive call, indented by depth; the sorted array; and the number of comparisons.

Constraints: No library sorting function. All array printing goes through `printArray`. The pivot is always the last element.

Assumptions: Values equal to the pivot go to the left side.

Edge Cases:

1. Already sorted array (worst case)
2. Duplicate values
3. Subarrays with 0 or 1 elements.

Designing Solution: The last number is the pivot. Partition moves every number that is less than or equal to the pivot to the left, then places the pivot right after them, which is its final spot. Quick Sort then sorts the left and right sides the same way. Each call prints its pivot and subarray, indented by depth, to show the recursion tree.

Pseudocode:

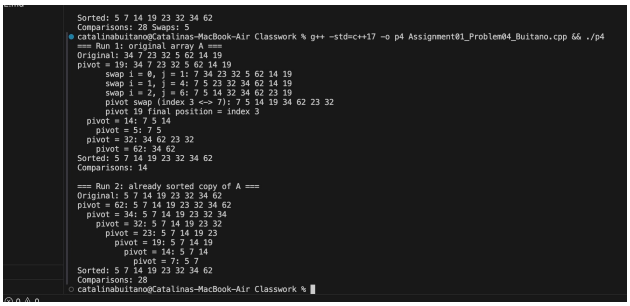


Figure 4: Given Array, Already Sorted Copy

Algorithm 1 Quick Sort (aka Lomuto partition, last element as pivot)

Input: An array of numbers *arr*, and indices *low* and *high*.
Output: $arr[low..high]$ sorted in increasing order.

procedure QUICKSORT($arr, low, high, comparisons, depth, traceFirst$)
  if $low < high$ **then**
    Print *depth* indents, "pivot =" $arr[high]$, PRINTARRAY($arr + low, high - low + 1$)
    $p \leftarrow$ PARTITION($arr, low, high, comparisons, traceFirst$ **and** $depth = 0$)
    QUICKSORT($arr, low, p - 1, comparisons, depth + 1,$ false)
    QUICKSORT($arr, p + 1, high, comparisons, depth + 1,$ false)
  end if
end procedure
procedure PARTITION($arr, low, high, comparisons, trace$)
  $pivot \leftarrow arr[high]$
  $i \leftarrow low - 1$
  for $j \leftarrow low$ **to** $high - 1$ **do**
    $comparisons \leftarrow comparisons + 1$
    if $arr[j] \leq pivot$ **then**
      $i \leftarrow i + 1$
      SWAP($arr[i], arr[j]$)
      if *trace* **then**
        Print "swap i =" i, "j =" j, PRINTARRAY($arr + low, high - low + 1$)
      end if
    end if
  end for
  SWAP($arr[i + 1], arr[high]$)
  if *trace* **then**
    Print "pivot swap (index" $i + 1$ "<->" *high* ")", PRINTARRAY($arr + low, high - low + 1$)
    Print "pivot" *pivot* "final position = index" $i + 1$
  end if
  return $i + 1$
end procedure

Screenshots:

Explanation: quickSort prints the pivot aka the last element and the subarray, indented by the depth of the recursion. Partition keeps i at the end of the section of numbers $\leq$ pivot, while j scans

from low to high -1 and counts each comparison. Every arr[j] $\leq$ pivot is swapped into that section, and for the first partition only, the code prints i, j, and the subarray after every swap. Finally, the pivot is swapped into position $i + 1$, and quickSort is called on the left and right sides with depth $+1$.

Correctness: After partition, every number left of the pivot is smaller than or equal to it and every number right of it is bigger, so the pivot is in its final spot. Each recursive call works on a smaller piece that leaves out the pivot, so the algorithm always finishes, and pieces with 0 or 1 elements are already sorted.

Time & Space Complexity: Best and average case $O(n \log n)$. Worst case $O(n^2)$ is on sorted input with the last-element pivot. Space $O(\log n)$ on average for the recursion, $O(n)$ in the worst case.

Run your program on an already sorted copy of A and count the comparisons again. Explain why a sorted array is the worst case when the last element is the pivot, and suggest one way to avoid it: On the sorted copy, my program makes 28 comparisons, compared with 14 on A. A sorted array is the worst case when the last element is the pivot, because the last element is always the largest number left. So every partition puts all the other numbers on one side and nothing on the other, and each call only removes one number which equals 28 comparisons, which is $O(n^2)$. One way to avoid this is to use the median of the first, middle, and last elements as the pivot, which picks the middle value on a sorted array and splits it evenly.

Problem 5: Merge Sort

Goal: Sort A into increasing order using Merge Sort.

Inputs: Array A with low = 0 and high = $n - 1$, plus a sorted copy and a reversed copy (62 34 32 23 19 14 7 5).

Outputs: Every subarray that is split, every merged subarray, the sorted array, the total comparisons, and the comparisons in the final merge.

Constraints: No library sorting function. All array printing goes through `printArray`. mid = (low + high) / 2.

Assumptions: Only comparisons made while merging are counted.

Edge Cases:

1. Already sorted array
2. Reversed array
3. Duplicate values
4. Subarrays with 0 or 1 elements.

Designing Solution: Split the array in half, sort each half, then merge them. Merging compares the front numbers of the two halves and takes the smaller one each time. When one half runs out, the rest of the other half is copied with no comparisons. `printArray` is called at every split and every merge.

Pseudocode:

Algorithm 2 Merge Sort

Input: An array of numbers *arr*, and indices *low* and *high*.
Output: The segment $arr[low..high]$ sorted in increasing order.

```
1: procedure MergeSort(arr, low, high, comparisons, finalMerge, verbose, depth)
2:     if low ≥ high then
3:         return
4:     end if
5:     if verbose then
6:         Print "split:", printArray(arr + low, high − low + 1)
7:     end if
8:     mid ← ⌊(low + high)/2⌋
9:     MergeSort(arr, low, mid, comparisons, finalMerge, verbose, depth + 1)
10:    MergeSort(arr, mid + 1, high, comparisons, finalMerge, verbose, depth + 1)
11:    before ← comparisons
12:    Merge(arr, low, mid, high, comparisons)
13:    if depth = 0 then
14:        finalMerge ← comparisons − before
15:    end if
16:    if verbose then
17:        Print "merged:", printArray(arr + low, high − low + 1)
18:    end if
19: end procedure
20: procedure Merge(arr, low, mid, high, comparisons)
21:    L ← arr[low..mid]
22:    R ← arr[mid + 1..high]
23:    i ← 0, j ← 0, k ← low
24:    while i < SizeOf(L) and j < SizeOf(R) do
25:        comparisons ← comparisons + 1
26:        if L[i] ≤ R[j] then
27:            arr[k] ← L[i]
28:            i ← i + 1
29:        else
30:            arr[k] ← R[j]
31:            j ← j + 1
32:        end if
33:        k ← k + 1
34:    end while
35:    while i < SizeOf(L) do
36:        arr[k] ← L[i]
37:        i ← i + 1, k ← k + 1
38:    end while
39:    while j < SizeOf(R) do
40:        arr[k] ← R[j]
41:        j ← j + 1, k ← k + 1
42:    end while
43: end procedure
```

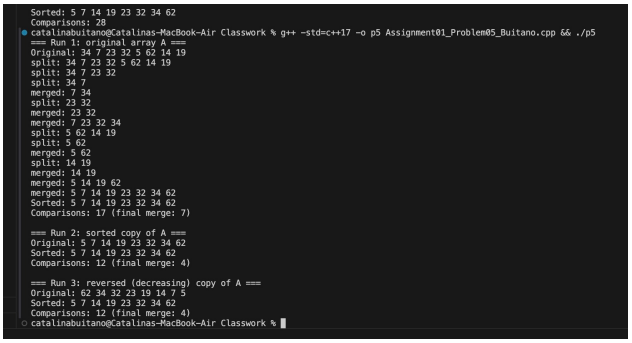


Figure 5: Given Array Input, Sorted Copy, Reversed Order

Screenshots:

Explanation: mergeSort returns right away when low $\geq$ high. Otherwise it prints the subarray being split, sets mid = (low + high)/2, sorts both halves recursively, and calls merge. Merge copies the halves into L and R, compares L[i] and R[j] one pair at a time, counts each comparison, and copies the smaller number back into the array. When one half runs out, the rest of the other half is copied with no comparisons. At depth 0, the code records how many comparisons the final merge made, and after every merge it prints the merged subarray.

Correctness: Merging two sorted halves always gives a sorted result, because each step takes the smallest remaining number from either half. The splits continue until each piece has 1 number, which is already sorted, so the merges build the whole array back up in order.

Time & Space Complexity: $O(n \log n)$ in the best, average, and worst case. Space complexity is $O(n)$ for the temporary arrays used in merging.

Count the total number of comparisons made while merging, and report how many of them happen in the final merge. Run your program on a sorted copy and on a reversed copy of A. Explain why the number of comparisons changes much less than for Bubble Sort or Quick Sort, and state how much extra memory Merge Sort needs: My program makes 17 comparisons on A, and 7 of them happen in the final merge. The sorted copy and the reversed copy both take 12 comparisons. That's a change of only 12 to 17, while Bubble Sort goes from 7 to 28 and Quick Sort goes from 14 to 28. Merge Sort always splits the array exactly in half no matter what order the numbers are in, so it always does the same amount of merging. The order only changes how soon one half runs out during a merge. Merge Sort needs $O(n)$ extra memory for the temporary arrays used while merging.

Problem 6: Heap Sort

Goal: Sort A into increasing order using Heap Sort.

Inputs: Array A ($n = 8$).

Outputs: The heap after the build phase, the heap part and sorted part after every pass, the sorted array, and the number of swaps inside siftDown.

Constraints: No library sorting function. All array printing goes through `printArray`. The heap is stored in the array itself.

Assumptions: Only swaps inside siftDown are counted, not the swaps that move the root to the

end.

Edge Cases:

1. Duplicate values (stability)
2. Array with 0 or 1 elements
3. A node with only one child

Designing Solution: First the array is turned into a max-heap by calling siftDown on every node that has children, from the last one up to the root. The largest number is then at the top. Each pass swaps the top to the end, and siftDown fixes the heap on the rest. siftDown moves a number down by swapping it with its bigger child until it's in the right place.

Pseudocode:

Algorithm 3 Heap Sort

Input: An array of numbers arr and its size n.
Output: arr sorted in increasing order.

```
1: procedure HeapSort(arr, n, swaps)
2:     swaps ← 0
3:     for k ← ⌊n/2⌋ − 1 downto 0 do
4:         SiftDown(arr, k, n, swaps)
5:     end for
6:     Print "Heap:", printArray(arr, n)
7:     for end ← n − 1 downto 1 do
8:         Swap(arr[0], arr[end])
9:         SiftDown(arr, 0, end, swaps)
10:        Print "end =" end, "heap:" printArray(arr, end), "sorted:" printArray(arr +
   end, n − end)
11:    end for
12: end procedure
13: procedure SiftDown(arr, root, size, swaps)
14:    while true do
15:        largest ← root, left ← 2 · root + 1, right ← 2 · root + 2
16:        if left < size and arr[left] > arr[largest] then
17:            largest ← left
18:        end if
19:        if right < size and arr[right] > arr[largest] then
20:            largest ← right
21:        end if
22:        if largest = root then
23:            break
24:        end if
25:        Swap(arr[root], arr[largest])
26:        swaps ← swaps + 1
27:        root ← largest
28:    end while
29: end procedure
```

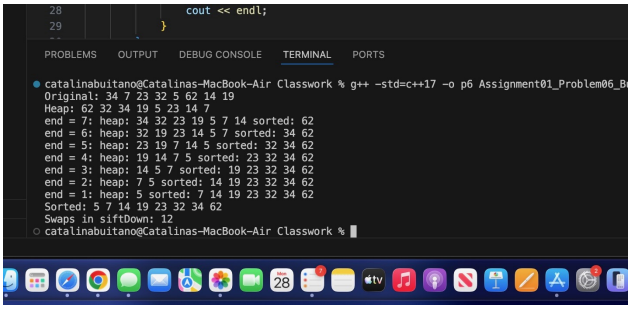
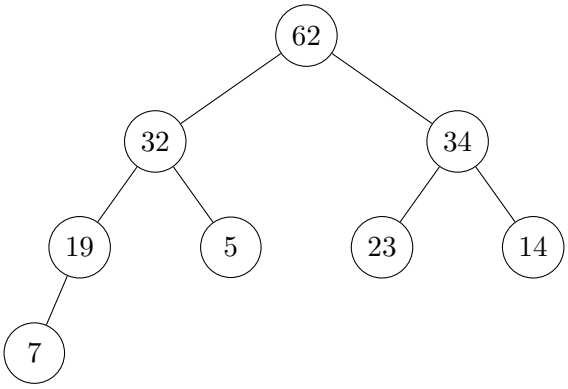


Figure 6: Heap Sort Array A Input

Screenshots:



Explanation: heapSort first calls siftDown on every node to build a max-heap, then prints the heap. Next, it swaps the the largest number with arr[end], calls siftDown on the first end numbers and then prints the heap part and the sorted part separately. siftDown compares a node with its left and right children when they exist, swaps it with the larger child, counts the swap, and repeats until the node is at least as big as both children.

Correctness: After the build phase, every parent is at least as big as its children, so the largest number is at the top. Each pass moves the top to the end, where it belongs, and siftDown restores the heap on the rest, so after $n - 1$ passes every number is in place.

Time & Space Complexity: $O(n \log n)$ in the best, average, and worst case. Space $O(1)$.

Is Heap Sort stable? Explain your answer in your report using a small example array that contains two equal values: Heap Sort is not stable. For example, in the array [5a, 5b, 3], where 5a and 5b are equal and 5a comes first, Heap Sort swaps the root 5a to the end first, then moves 5b to the top and swaps it into place, giving 3, 5b, 5a. The two equal 5s end up in the opposite order from how they started, because Heap Sort moves the root straight to the end of the array without checking where equal values began.

Problem 7: All 6 Algorithms Implementation

Goal: Compare all six algorithms in one program and check them against the theory.

Inputs: A, a sorted copy of A, a reversed copy of A (62 34 32 23 19 14 7 5), and random arrays of 1,000, 5,000, and 10,000 integers between 1 and 100,000.

Outputs: A comparison table, a timing table, and the first 10 elements of every sorted array.

Constraints: Step-by-step printing turned off. `<chrono>` for timing, `<random>` for values, and a fresh copy of the same data for every algorithm.

Assumptions: Comparisons are counted with long long, since the n^2 algorithms make about 50 million at $n = 10{,}000$.

Edge Cases:

1. Sorted input (best case for Bubble and Insertion, worst case for Quick)
2. reversed input (worst case for Bubble and Insertion)
3. duplicate values in the random data.

Designing Solution: All six sorts are in one file with the printing removed, and each one counts comparisons. One random array is generated, and each algorithm sorts a fresh copy of the same data. Each sort is timed with `<chrono>`, and `printArray` prints the first 10 elements to check the result.

Pseudocode:

Algorithm 4 Compare all six sorting algorithms

Input: Arrays A, *sortedA*, *reversedA* of size $n = 8$, and random arrays of sizes 1000, 5000, 10000.
Output: Comparison table, first 10 elements of each sorted array, and timing table.

```
algs ← {Bubble, Insertion, Selection, Quick, Merge, Heap}
n ← 8
inputs ← {A, sortedA, reversedA}
for each alg in algs do
    Print alg name
    for t ← 0 to 2 do
        work ← copy of inputs[t]
        comps ← 0
        alg(work, n, comps)
        Print comps
    end for
end for
sizes ← {1000, 5000, 10000}
randomData ← 10000 random integers from 1 to 100000 using gen and dist
for s ← 0 to 2 do
    size ← sizes[s]
    Print "n =" size
    for a ← 0 to 5 do
        work ← copy of first size values of randomData
        c ← 0
        t0 ← now()
        algs[a](work, size, c)
        t1 ← now()
        times[a][s] ← t1 − t0                          ▷ in microseconds
        Print algs[a] name, printArray(work, 10)
    end for
end for
for a ← 0 to 5 do
    Print algs[a] name, times[a][s]/1000 for each s    ▷ in milliseconds
end for
```

Screenshots:

Explanation: All six sorts are copied from Problems 1–6 with the printing removed, and each one counts comparisons passed by reference. They are stored in one list so a single loop can run them all. The first part runs each algorithm on fresh copies of A, a sorted copy, and a reversed copy, and prints the comparison table. The 2nd part generates 10,000 random numbers with a fixed seed (42). For each size, every algorithm sorts a fresh copy of the first n values, is timed with chrono, and has its first 10 elements printed with printArray. The program ends by printing the timing table in milliseconds.

Correctness: Every algorithm sorts a fresh copy of the same data, so the comparison is fair. `printArray` shows the same first 10 numbers in increasing order for all six algorithms at every size, which confirms each sort works.

Time & Space Complexity: Bubble, Insertion, and Selection are $O(n^2)$ in the worst case. Merge

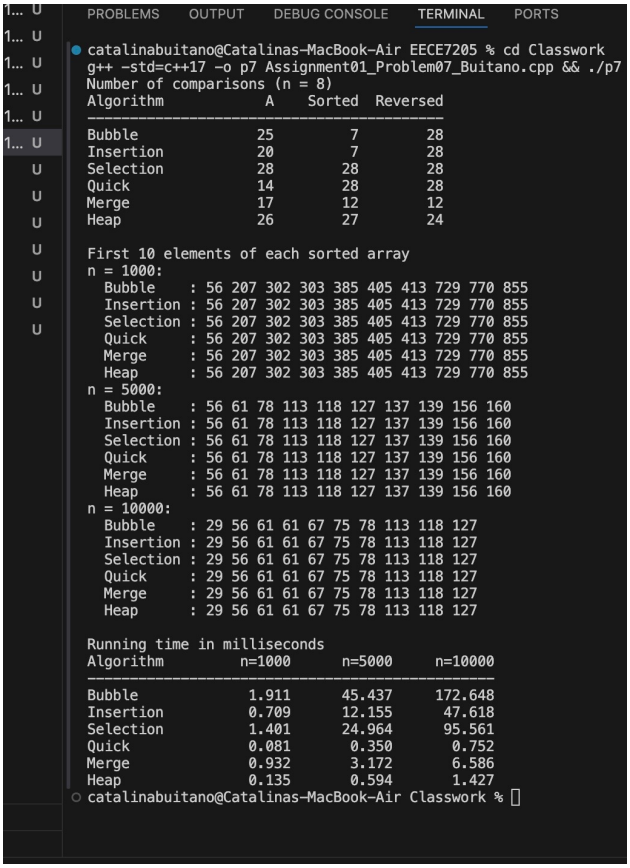


Figure 7: Given A, Sorted, Reversed, First 10 Elements, and Running Time Tests

and Heap are $O(n \log n)$ in every case. Quick is $O(n \log n)$ on average and $O(n^2)$ in the worst case. Merge Sort uses $O(n)$ extra space; the others sort in place.

Make a table of the number of comparisons each algorithm makes on three inputs: A, a sorted copy of A, and a reversed copy of A:

The table below shows the number of comparisons each algorithm made on A, a sorted copy of A, and a reversed copy of A.

Algorithm	A	Sorted	Reversed
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	26	27	24

Measure how long each algorithm takes to sort random arrays of 1,000, 5,000, and 10,000 integers between 1 and 100,000. Use `<chrono>` for timing and `<cstdlib>` (rand) or `<random>` for the values, and copy the same random data into a fresh array for every algorithm. Present the results as a table or a plot: The random values were generated with `<random>` (`mt19937` with seed 42 and `uniform_int_distribution` from 1 to 100,000), and each algorithm sorted a fresh copy of the same data. Each sort was timed with `<chrono>`. The running times in milliseconds are shown below.

Algorithm	n = 1,000	n = 5,000	n = 10,000
Bubble Sort	1.911	45.437	172.648
Insertion Sort	0.709	12.155	47.618
Selection Sort	1.401	24.964	95.561
Quick Sort	0.081	0.350	0.752
Merge Sort	0.932	3.172	6.586
Heap Sort	0.135	0.594	1.427

Call printArray on the first 10 elements of each sorted array to check that the sorting is correct: After each sort, `printArray` was called on the first 10 elements of the sorted array. For every size, all six algorithms printed the same 10 values in increasing order, which confirms that every sort is correct.

Size	First 10 elements (same for all six algorithms)
n = 1,000	56 207 302 303 385 405 413 729 770 855
n = 5,000	56 61 78 113 118 127 137 139 156 160
n = 10,000	29 56 61 61 67 75 78 113 118 127

Which algorithms grow like n^2 and which grow like $n \log n$? Do your measurements agree with the theory? Which algorithm would you choose for a large random array, and which for an array that is already almost sorted? Explain your choices: Bubble Sort, Insertion Sort, and Selection Sort grow like n^2, while Quick Sort, Merge Sort, and Heap Sort

grow like $n \log n$. My measurements agree with the theory and when n doubled from 5,000 to 10,000, the n^2 algorithms took about 4 times as long, while the $n \log n$ algorithms took only about 2 times as long and stayed much faster at every size. For a large random array, I would choose Quick Sort, because it was the fastest in my tests and sorts in place without extra memory. For an array that is already almost sorted, I would choose Insertion Sort, because each number is already close to its correct spot and only has to shift a few positions, so it runs close to $O(n)$.